

Se evidencia un cambio drástico, en el cual se observa que agregue un buscador en el cual filtra por categorías de productos, el diseño quería implementarlo como la tienda de referencia de Falabella, en la cual los productos se muestren todos, pero después los filtra, además hice el cambio de imagen de esta vista por una más estética y de acuerdo a la vista, por supuesto no muestra imágenes porque eso ya es de manera interna.

Documentación del Componente:

Productos

1. Descripción General

El componente **Productos** es el encargado de mostrar el catálogo principal de productos de A&L Compresores y Partes.

Permite a los usuarios:

- Consultar productos disponibles.
- Buscar productos por nombre o referencia.
- Filtrar productos por categorías.
- Filtrar por marcas.
- Ordenar resultados por precio.
- Visualizar imágenes del producto.
- Acceder al detalle completo de cada artículo.

La información es obtenida dinámicamente desde el servidor mediante una API REST.

2. Objetivo

Facilitar la exploración y búsqueda de productos industriales mediante una interfaz intuitiva, rápida y adaptable a dispositivos móviles y de escritorio.

3. Funcionalidades Principales

Consulta de Productos

Al cargar la vista, el sistema realiza una petición al backend para obtener todos los productos registrados.

Posteriormente:

- Procesa la información recibida.
 - Organiza las imágenes.
 - Prepara los datos para su visualización.
-

Sistema de Búsqueda

Permite encontrar productos utilizando:

- Nombre del producto.
- Referencia interna.

La búsqueda se realiza en tiempo real sin necesidad de recargar la página.

Filtrado por Marca

El usuario puede visualizar únicamente los productos pertenecientes a una marca específica.

Las marcas se generan automáticamente a partir de la información almacenada en la base de datos.

Filtrado por Tipo

Los productos son clasificados automáticamente en categorías:

- Filtros
 - Lubricantes
 - Compresores
 - Válvulas
 - Herramientas
 - Accesorios
 - Otros
-

Filtrado por Subtipo

Dependiendo del tipo seleccionado, se habilitan filtros más específicos.

Filtros

- Separador
- Aceite
- Aire
- Otros

Lubricantes

- Cuarto
 - Galón
 - Garrafa
 - Otros
-

Ordenamiento

Los productos pueden organizarse por:

- Relevancia
 - Menor precio
 - Mayor precio
-

Visualización de Imágenes

Cada producto dispone de un mini carrusel que permite:

- Visualizar múltiples imágenes.
 - Navegar entre fotografías.
 - Mostrar indicadores visuales de posición.
-

Optimización de Imágenes

Las imágenes almacenadas en Cloudinary son optimizadas automáticamente para:

- Reducir tiempo de carga.
 - Disminuir consumo de datos.
 - Mejorar experiencia móvil.
-

Navegación al Detalle del Producto

Al seleccionar "Ver Detalles", el sistema:

- 1. Guarda el identificador del producto.
- 2. Cambia a la vista de detalle.
- 3. Carga toda la información técnica y comercial.

4. Componentes Utilizados

Componente	Función
Navbar	Barra de navegación principal
Footer	Pie de página
LoginModal	Inicio de sesión
MiniCarruselCatalog	Galería resumida de imágenes

5. Estados del Componente

Estado	Función
showModal	Controla la ventana de login
marcaActiva	Marca seleccionada
tipoActivo	Categoría seleccionada
subtipoActivo	Subcategoría seleccionada
busqueda	Texto de búsqueda
ordenamiento	Tipo de orden aplicado
productosApi	Productos obtenidos desde la API
loading	Estado de carga

6. Flujo de Funcionamiento

Paso 1

El usuario accede al catálogo.

Paso 2

El sistema consulta la API de productos.

Paso 3

Los datos recibidos son procesados.

Paso 4

Se generan automáticamente:

- Categorías.
- Marcas.
- Subcategorías.

Paso 5

El usuario aplica filtros o búsquedas.

Paso 6

La lista se actualiza dinámicamente.

Paso 7

El usuario puede acceder al detalle del producto.

7. Comunicación con el Backend

Endpoint Principal

GET /productos

Propósito

Obtener todos los productos disponibles para el catálogo.

Respuesta Esperada

```
[
{
  "id": 1,
  "nombre": "Filtro Separador",
  "marca": "Atlas Copco",
  "precio": 120000,
  "stock": 15
}
```

8. Clasificación Automática de Productos

La clasificación se realiza mediante palabras clave encontradas en:

- Nombre
- Descripción
- Referencia interna

Ejemplo:

Palabra Detectada	Categoría
filtro	Filtros
aceite	Lubricantes
compresor	Compresores
válvula	Válvulas
herramienta	Herramientas

9. Requerimientos Funcionales

RF-01

El sistema debe mostrar el catálogo completo de productos.

RF-02

El sistema debe permitir realizar búsquedas.

RF-03

El sistema debe permitir filtrar por categoría.

RF-04

El sistema debe permitir filtrar por marca.

RF-05

El sistema debe permitir ordenar los productos.

RF-06

El sistema debe mostrar imágenes optimizadas.

RF-07

El sistema debe permitir acceder al detalle del producto.

10. Requerimientos No Funcionales

RNF-01

La interfaz debe ser responsive.

RNF-02

Las imágenes deben estar optimizadas para rendimiento.

RNF-03

La búsqueda debe ejecutarse en tiempo real.

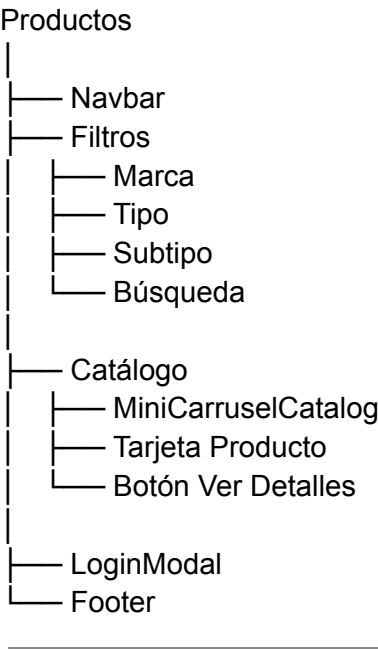
RNF-04

La navegación debe realizarse sin recargar la página.

RNF-05

El sistema debe soportar grandes volúmenes de productos.

11. Arquitectura del Componente



12. Resultado Esperado

El componente Productos proporciona una experiencia eficiente para la consulta de artículos industriales, permitiendo localizar productos rápidamente mediante filtros inteligentes, búsqueda dinámica y visualización optimizada de imágenes, mejorando significativamente la experiencia de compra del usuario.


```

import { useState, useEffect } from "react";
//import { supabase } from "../lib/client.js";
import Navbar from "../components/Home/Navbar";
import Footer from "../components/Home/Footer";
import LoginModal from "../components/Login/LoginModal";

const TIPOS_PRINCIPALES = ["Todos", "Filtros", "Lubricantes",
"Compresores", "Válvulas", "Herramientas", "Accesorios", "Otros"];

const SUBTIPOS_POR_TIPO = {
  Filtros: ["Todos", "Separador", "Aceite", "Aire", "Otros"],
  Lubricantes: ["Todos", "Cuarto", "Galon", "Garrafa", "Otros"]
};

const obtenerTipoPrincipal = (producto) => {
  const texto = `${producto.nombre || ""} ${producto.descripcion || ""}
${producto.referencia_interna || ""}`.toLowerCase();
  if (texto.includes("filtro")) return "Filtros";
  if (texto.includes("lubricante") || texto.includes("aceite") ||
texto.includes("galon") || texto.includes("galón") ||
texto.includes("garrafa")) return "Lubricantes";
  if (texto.includes("compresor")) return "Compresores";
  if (texto.includes("válvula") || texto.includes("valvula")) return
"Válvulas";
  if (texto.includes("herramienta")) return "Herramientas";
  if (texto.includes("accesorio")) return "Accesorios";
  return "Otros";
};

const obtenerSubtipo = (producto, tipoPrincipal) => {
  const texto = `${producto.nombre || ""} ${producto.descripcion || ""}
${producto.referencia_interna || ""}`.toLowerCase();
  if (tipoPrincipal === "Filtros") {
    if (texto.includes("separador")) return "Separador";
    if (texto.includes("aceite")) return "Aceite";
    if (texto.includes("aire")) return "Aire";
    return "Otros";
  }
  if (tipoPrincipal === "Lubricantes") {
    if (texto.includes("cuarto")) return "Cuarto";
    if (texto.includes("galon") || texto.includes("galón")) return
"Galon";
    if (texto.includes("garrafa")) return "Garrafa";
  }

```

```

        return "Otros";
    }
    return "Otros";
};

const optimizarUrlCloudinary = (url, opciones = {}) => {
    if (!url || !url.includes("cloudinary.com")) return url;

    const {
        width = 500,
        quality = "auto",
        format = "auto",
    } = opciones;

    const partes = url.split("/upload/");
    if (partes.length !== 2) return url;

    const transformaciones =
`w_${width},q_${quality},f_${format},c_limit`;
    return `${partes[0]}/upload/${transformaciones}/${partes[1]}`;
};

function MiniCarruselCatalog({ imagenes = [], nombreProducto }) {
    const [indexActual, setIndexActual] = useState(0);
    const imgUrl =
optimizarUrlCloudinary(imagenes[indexActual]?.imagen_url ||
"https://res.cloudinary.com/ddyrqgdxq/image/upload/v1780240514/IMG_Prod
uctos.webp", { width: 500 });

    const anteriorImagen = (e) => {
        e.stopPropagation();
        setIndexActual((prev) => (prev === 0 ? imagenes.length - 1 : prev -
1));
    };

    const gSiguieteImagen = (e) => {
        e.stopPropagation();
        setIndexActual((prev) => (prev === imagenes.length - 1 ? 0 : prev +
1));
    };

    return (

```

```

    <div className="position-relative overflow-hidden group-carrusel"
style={{ height: "180px", backgroundColor: "#fff" }}>
    <div className="p-3 text-center bg-white d-flex
align-items-center justify-content-center h-100">
        <img
            src={imgUrl}
            className="img-fluid h-100 object-fit-contain"
            alt={` ${nombreProducto} - vista ${indexActual + 1}`}
            loading="lazy"
            onMouseEnter={() => {
                if (imagenes.length > 1) {
                    const nextIdx = (indexActual + 1) % imagenes.length;
                    const img = new Image();
                    img.src =
optimizarUrlCloudinary(imagenes[nextIdx]?.imagen_url, { width: 500 });
                }
            }}
        />
    </div>

    {imagenes.length > 1 && (
        <>
            <button
                onClick={anteriorImagen}
                className="btn-carrusel-nav position-absolute start-0
top-50 translate-middle-y ms-2"
                type="button"
                aria-label="Imagen anterior"
            >
                <i className="bi bi-chevron-left"></i>
            </button>
            <button
                onClick={gSiguienteImagen}
                className="btn-carrusel-nav position-absolute end-0 top-50
translate-middle-y me-2"
                type="button"
                aria-label="Siguiente imagen"
            >
                <i className="bi bi-chevron-right"></i>
            </button>
        </>
    )

```

```

        <div className="position-absolute bottom-0 start-50
translate-middle-x mb-2 d-flex gap-1 bg-dark bg-opacity-25 rounded-pill
px-2 py-1">
            {imagenes.map((_, idx) => (
                <span
                    key={idx}
                    onClick={(e) => { e.stopPropagation();
setIndexActual(idx); }}
                    className="rounded-circle cursor-pointer"
                    style={{
                        width: "6px",
                        height: "6px",
                        backgroundColor: idx === indexActual ? "#ffffff" :
"rgba(255, 255, 255, 0.5)",
                        transition: "0.2s"
                    }}
                />
            ))}
        </div>
    </>
    )}
</div>
);
}

function Productos({
    setVista,
    usuario,
    logout,
    login,
    setProductoSeleccionadoId,
    totalItems,
    setCartOpen,
    onOpenLogin
}) {
    const [showModal, setShowModal] = useState(false);
    const [marcaActiva, setMarcaActiva] = useState("");
    const [tipoActivo, setTipoActivo] = useState("Todos");
    const [subtipoActivo, setSubtipoActivo] = useState("Todos");
    const [busqueda, setBusqueda] = useState("");
    const [ordenamiento, setOrdenamiento] = useState("Relevancia");
    const [productosApi, setProductosApi] = useState([]);
    const [loading, setLoading] = useState(true);

```

```

useEffect(() => {
  async function cargarProductos() {
    setLoading(true);
    try {
      // 1. Hacemos la petición a tu API de Node.js en el puerto 3001
      const response = await
fetch("http://localhost:3001/productos");

      if (!response.ok) {
        throw new Error(`Error en el servidor: ${response.status}`);
      }

      const data = await response.json();

      // 2. Procesamos las imágenes exactamente igual a como lo
tenías
      const productosProcesados = (data || []).map(p => {
        const imagenesOrdenadas = p.producto_imagenes?.sort((a, b) =>
{
          if (a.es_principal) return -1;
          if (b.es_principal) return 1;
          return (a.orden || 0) - (b.orden || 0);
        }) || [];

        return {
          ...p,
          todas_las_imagenes: imagenesOrdenadas
        };
      });

      setProductosApi(productosProcesados);
    } catch (err) {
      console.error("Error al conectar con el Backend:",
err.message);
    } finally {
      setLoading(false);
    }
  }

  cargarProductos();
}, []);

```

```

const marcasDisponibles = Array.from(
  new Set(productosApi.map((p) => (p.marca ||
    "").trim()).filter(Boolean))
).sort((a, b) => a.localeCompare(b, "es", { sensitivity: "base" }));

// Proceso unificado de filtrado y ordenamiento con CORRECCIÓN DE
REINICIO DE BÚSQUEDA
const productosFiltradosYOrdenados = productosApi
  .filter((p) => {
    const marcaProducto = (p.marca || "").trim();
    const tipoProducto = obtenerTipoPrincipal(p);
    const subtipoProducto = obtenerSubtipo(p, tipoProducto);

    const cumpleMarca = !marcaActiva || marcaActiva ===
marcaProducto;
    const cumpleTipo = tipoActivo === "Todos" || tipoProducto ===
tipoActivo;
    const cumpleSubtipo = subtipoActivo === "Todos" ||
subtipoProducto === subtipoActivo;

    // CORRECCIÓN AQUÍ: .trim() limpia espacios fantasmas y evalúa
correctamente si está vacío para restablecer todo
    const busquedaLimpia = busqueda.trim().toLowerCase();
    const cumpleBusqueda =
      busquedaLimpia === "" ||
      (p.nombre && p.nombre.toLowerCase().includes(busquedaLimpia))
||
      (p.referencia_interna &&
p.referencia_interna.toLowerCase().includes(busquedaLimpia));

    return cumpleMarca && cumpleTipo && cumpleSubtipo &&
cumpleBusqueda;
  })
  .sort((a, b) => {
    if (ordenamiento === "Menor precio") {
      return Number(a.precio) - Number(b.precio);
    }
    if (ordenamiento === "Mayor precio") {
      return Number(b.precio) - Number(a.precio);
    }
    return 0;
  });

```



```

<aside className="col-lg-3 d-none d-lg-block">
  <div className="card border-0 shadow-sm p-4 sticky-top"
style={{ top: "100px", borderRadius: "15px" }}>
    <h5 className="fw-bold mb-4">Filtrar por</h5>

    <div className="mb-4">
      <label htmlFor="busquedaInput" className="form-label
small fw-bold text-muted text-uppercase">Buscador</label>
      <div className="input-group border rounded-pill
overflow-hidden">
        <span className="input-group-text bg-white border-0"
aria-hidden="true"><i className="bi bi-search"></i></span>
        <input
          id="busquedaInput"
          type="text"
          className="form-control border-0 shadow-none"
          placeholder="¿Qué buscas?"
          value={busqueda}
          onChange={(e) => setBusqueda(e.target.value)} //
Reacciona de inmediato en tiempo real
          aria-label="Buscar productos"
        />
        {/* Botón de limpiar sutil para mejorar la
experiencia de usuario si queda atascado */}
        {busqueda && (
          <button
            className="btn btn-link text-muted border-0
bg-transparent pe-3 y-0 shadow-none text-decoration-none"
            onClick={() => setBusqueda("")}
            style={{ fontSize: "0.85rem" }}
            type="button"
          >
            <i className="bi bi-x-circle-fill"></i>
          </button>
        )}
      </div>
    </div>

    <div className="mb-4">
      <label className="form-label small fw-bold text-muted
text-uppercase">Marca</label>
      <select

```



```

        className="form-select"
        value={marcaActiva}
        onChange={(e) => setMarcaActiva(e.target.value)}
      >
      <option value="">Todas las marcas</option>
      {marcasDisponibles.map((marca) => (
        <option key={marca} value={marca}>{marca}</option>
      ))}
    </select>
  </div>

  <div className="mb-4">
    <label className="form-label small fw-bold text-muted
text-uppercase">Tipo</label>
    <select
      className="form-select"
      value={tipoActivo}
      onChange={(e) => {
        setTipoActivo(e.target.value);
        setSubtipoActivo("Todos");
      }}
    >
      {TIPOS_PRINCIPALES.map((tipo) => (
        <option key={tipo} value={tipo}>{tipo}</option>
      ))}
    </select>
  </div>

  {(tipoActivo === "Filtros" || tipoActivo ===
"Lubricantes") && (
    <div className="mb-4">
      <label className="form-label small fw-bold text-muted
text-uppercase">Subtipo</label>
      <select
        className="form-select"
        value={subtipoActivo}
        onChange={(e) => setSubtipoActivo(e.target.value)}
      >
        {SUBTIPOS_POR_TIPO[tipoActivo].map((subtipo) => (
          <option key={subtipo}
value={subtipo}>{subtipo}</option>
        ))}
      </select>
    </div>
  )}

```

```

        </div>
    )}

    <div className="bg-light p-3 rounded-4 mt-4">
        <p className="small mb-0 text-dark">
            <i className="bi bi-truck me-2 text-warning"></i>
            Envío gratis en Bogotá por compras superiores a
$500.000
        </p>
    </div>
</div>
</aside>

<main className="col-lg-9">
    <div className="d-flex justify-content-between
align-items-center mb-4 px-2">
        <p className="text-muted mb-0">Mostrando
<b>{productosFiltradosYOrdenados.length}</b> productos</p>
        <select
            className="form-select w-auto border-0 shadow-sm
rounded-pill"
            style={{ fontSize: "0.9rem" }}
            value={ordenamiento}
            onChange={(e) => setOrdenamiento(e.target.value)}
        >
            <option value="Relevancia">Ordenar por:
Relevancia</option>
            <option value="Menor precio">Menor precio</option>
            <option value="Mayor precio">Mayor precio</option>
        </select>
    </div>

    {loading ? (
        <div className="row g-4">
            {[1, 2, 3, 4, 5, 6].map((n) => (
                <div key={n} className="col-6 col-md-4">
                    <div className="card border-0 placeholder-glow
h-100 p-4" style={{ borderRadius: "16px", minHeight: "340px" }}>
                        <div className="placeholder col-12 mb-3
bg-secondary opacity-10" style={{ height: "160px", borderRadius: "12px"
}}></div>
                        <div className="placeholder col-8 mb-2
bg-secondary opacity-15"></div>

```

```

        <div className="placeholder col-5 bg-secondary
opacity-15"></div>
      </div>
    </div>
  )})
</div>
) : (
  <div className="row g-3 g-md-4">
    {productosFiltradosYOrdenados.map((p) => (
      <div key={p.id} className="col-6 col-md-4 col-xl-3">
        <div className="card h-100 border-0 shadow-sm
product-card transition-all"
          style={{ borderRadius: "16px", overflow:
"hidden", background: "#fff" }}>

          <span className="position-absolute badge
rounded-pill bg-dark mt-2 ms-2 z-1 text-uppercase"
            style={{ fontSize: "0.60rem" }}>
            {p.marca || "Industrial"}
          </span>

          <MiniCarruselCatalog
            imagenes={p.todas_las_imagenes}
            nombreProducto={p.nombre}
          />

          <div className="card-body p-2 p-md-3 d-flex
flex-column">
            <h6 className="fw-bold mb-1 text-truncate-2
text-dark"
              style={{ height: "40px", fontSize: "0.85rem"
            }}>
              {p.nombre}
            </h6>
            <p className="text-muted small mb-2
text-truncate">Ref: {p.referencia_interna || "N/A"}</p>

            <div className="mt-auto">
              <div className="mb-2">
                <span className="h6 fw-bold text-dark"
style={{ fontSize: "0.9rem" }}>

```

```

    ${Number(p.precio).toLocaleString('es-CO', { minimumFractionDigits: 0
    })} COP

        </span>
    </div>
    <button
        className="btn btn-warning w-100
rounded-pill fw-bold btn-details d-flex align-items-center
justify-content-center gap-1"
        style={{ fontSize: "0.8rem", padding: "8px"
    }}

        onClick={() => handleVerDetalles(p.id)}
    >
        Ver Detalles <i className="bi
bi-arrow-right-short fs-6"></i>
    </button>
</div>
</div>
</div>
</div>
    )}
</div>
)}

    {productosFiltradosYOrdenados.length === 0 && !loading && (
        <div className="text-center py-5">
            <i className="bi bi-search display-1 text-muted"></i>
            <h4 className="mt-3">No encontramos productos</h4>
            <p className="text-muted">Prueba con otra categoría o
palabra clave.</p>
        </div>
    )}
</main>
</div>
</div>

    <Footer setVista={setVista} onAdminLogin={() =>
setShowModal(true)} />
    {showModal && <LoginModal login={login} onClose={() =>
setShowModal(false)} />}

    <style>{`

```

```

        .product-card { transition: transform 0.3s cubic-bezier(0.4, 0,
0.2, 1), opacity 0.3s; }
        .product-card:hover { transform: translateY(-6px); box-shadow:
0 15px 30px rgba(0,0,0,0.06) !important; }
        .hover-bg-light:hover { background-color: #f1f3f5; }
        .text-truncate-2 { display: -webkit-box; -webkit-line-clamp: 2;
-webkit-box-orient: vertical; overflow: hidden; }
        .cursor-pointer { cursor: pointer; }
        .btn-details { transition: 0.2s; border: none;
background-color: #ffc107; color: #212529; }
        .btn-details:hover { background-color: #10142D; color: white; }
        .btn-carrusel-nav {
            background: rgba(255, 255, 255, 0.9);
            border: none;
            border-radius: 50%;
            width: 30px;
            height: 30px;
            display: flex;
            align-items: center;
            justify-content: center;
            box-shadow: 0 2px 6px rgba(0,0,0,0.15);
            color: #10142D;
            font-size: 11px;
            transition: transform 0.2s ease, opacity 0.2s ease;
            opacity: 0;
            z-index: 3;
        }
        .group-carrusel:hover .btn-carrusel-nav { opacity: 1; }
        .btn-carrusel-nav:hover {
            background: #ffc107;
            color: #212529;
            transform: scale(1.08);
        }
    `}</style>
</div>
);
}

export default Productos;

```